

YesGo AI 数字员工平台

需求与架构文档 v2.0

定位：垂直医药 B2B SaaS

对标：Marvis 多智能体可视化协作平台

当前版本：v0.3.0 | 日期：2026-07-24

本文档为 YesGo 平台完整需求与架构设计文档，涵盖三层架构、技术栈、

YesGo AI 数字员工平台——需求与架构文档 v2.0

版本：v2.0 | 日期：2026-07-24

对标产品：Marvis (腾讯多智能体可视化协作平台)

行业定位：垂直医药 B2B SaaS

当前版本：v0.3.0 (前后端数据通讯闭环已打通)

目录

- [产品概述](#1-产品概述) 2. [技术架构总览](#2-技术架构总览) 3. [技术栈详情](#3-技术栈详情) 4. [项目结构](#4-项目结构) 5. [第二层：天网大脑后端](#5-第二层天网大脑后端) 6. [第三层：多商户前端](#6-第三层多商户前端) 7. [智能体体系](#7-智能体体系) 8. [视图功能清单](#8-视图功能清单) 9. [数据模型](#9-数据模型) 10. [API 接口设计](#10-api-接口设计) 11.

1. 产品概述

YesGo 是一款面向医药行业的 AI 数字员工桌面应用 (Electron 桌面客户端)。底层对接多租户医药 SaaS 数据底座 (商品/库存/订单/流向/客户)，上层由**统筹中控 A Agent** 统一调度 5 个垂直业务智能体，通过自然语言对话驱动业务闭环。

****核心设计理念****：一句话派发任务，智能体自动干完活回来汇报。

1.1 用户体验对标

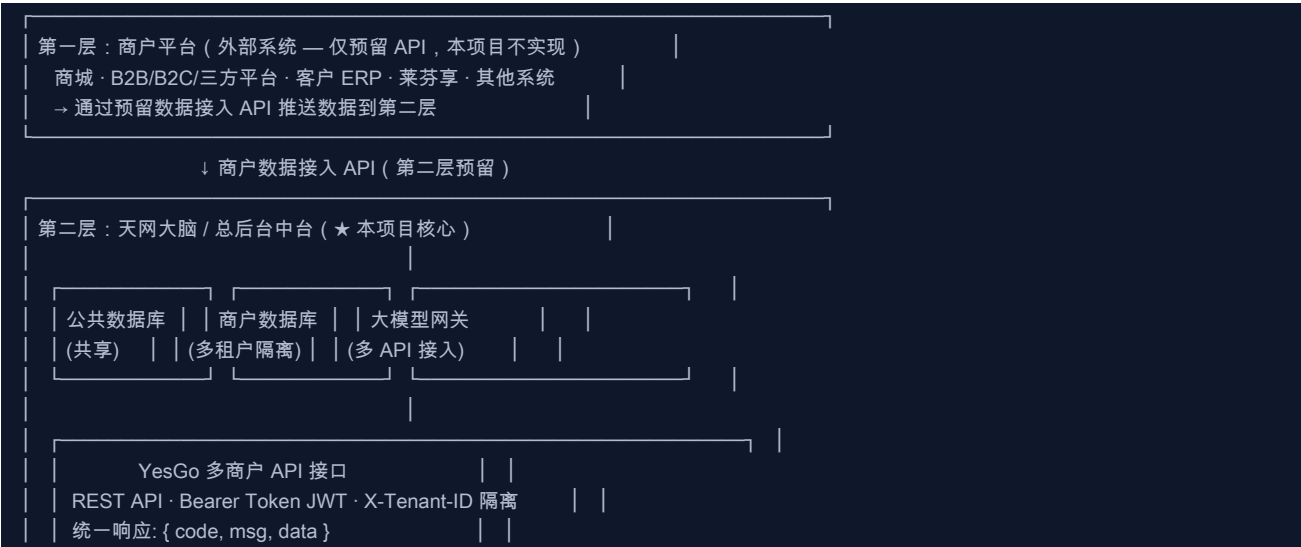
维度	Marvis (腾讯)	YesGo
标志物	牛	白兔
形态	桌面 App	桌面 App (Electron)
核心	多智能体工位可视化协作	中控 A + 5 业务智能体工位
入口	自然语言对话	对话工作台 / 办公室输入条
行业	通用	垂直医药 B2B

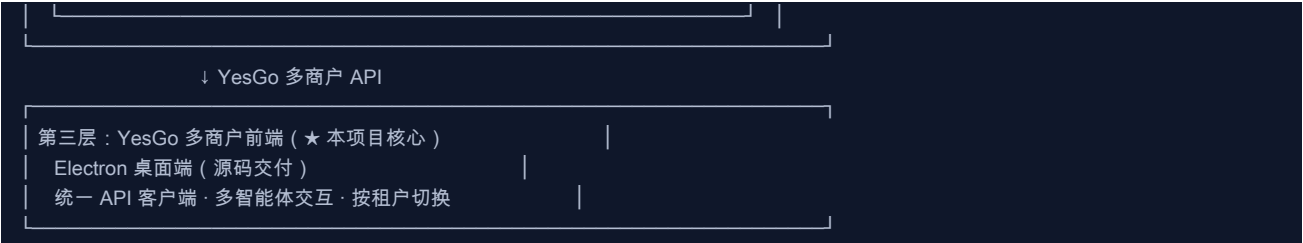
1.2 交付形态

形态	定位	交付方式
SaaS 版	多租户共享平台，开箱即用	云服务，按租户订阅
独立版	单租户私有化部署	源码交付 (含第二层后端 + 第三层前端)

2. 技术架构总览

2.1 三层架构





2.2 核心设计原则

- ****中台集中****：所有 AI、数据、配置能力在第二层统一管理
- 2. ****终端轻量****：第三层不存储业务数据，仅通过 API 与中台交互
- 3. ****商户隔离****：数据库按 ``tenant_{credit_code}`` 严格隔离
- 4. ****模型可替换****：统一网关抽象，切换模型只改 Provider
- 5. ****API 优先****：第三层所有功能通过第二层 YesGo 多商户 API 实现
- 6. ****源码可交付****：独立版保证可私有化部署，不依赖平台 SaaS

3. 技术栈详情

3.1 第二层：天网大脑后端

层	技术选型	说明
后端框架	Django 6.0 + Django REST Framework	Python 3.14
数据库	SQLite (开发) → PostgreSQL (生产)	多 schema 租户隔离
认证	djangorestframework-simplejwt	Bearer Token + 自动刷新
跨域	django-cors-headers	CORS_ALLOW_ALL_ORIGINS
智能体引擎	Dify (5 个平台级工作流，独立 API Key)	预留接口
消息队列	Kafka	预留
对象存储	阿里云 OSS	预留
容器化	Docker Compose	生产部署

3.2 第三层：多商户前端

层	技术选型	版本
桌面壳	Electron	33
构建工具	Vite	5
UI 框架	React	18
开发语言	TypeScript	5.x
样式	Tailwind CSS	3 (darkMode: class + 自定义 CSS 变量)
图标	lucide-react	最新
状态管理	React Context + useReducer	—
路由	单页组件切换 (ViewKey 枚举)	—
打包 (macOS)	electron-builder → DMG / ZIP	—
打包 (Windows)	electron-builder → NSIS	—

4. 项目结构

4.1 完整目录树

```
YesGo/
├── yesgo-backend/           # 第二层：天网大脑 ( Django )
│   ├── manage.py           # Django 管理入口
│   ├── config/
│   │   ├── settings.py     # 全局配置 ( CORS/JWT/数据库 )
│   │   ├── urls.py         # API 根路由 ( 8 模块注册 )
│   │   ├── wsgi.py
│   │   └── asgi.py
│   ├── apps/
│   │   ├── platform/       # 平台管理
│   │   │   ├── views.py    # 租户/成员/角色/配置 CRUD
│   │   │   ├── urls_auth.py # /api/v1/auth/* ( 登录/当前用户 )
│   │   │   ├── urls_tenant.py # /api/v1/tenant/* ( 租户/成员/角色 )
│   │   │   ├── urls_config.py # /api/v1/config/* ( 系统配置/Dify )
│   │   │   ├── urls_health.py # /api/v1/health/*
│   │   │   └── models.py
│   │   ├── chat/           # 智能体对话
│   │   │   ├── views.py    # 消息发送/历史/会话
│   │   │   └── urls.py
│   │   ├── dashboard/      # 经营看板
│   │   │   ├── views.py    # 总览/KPI/预警
│   │   │   └── urls.py
│   │   ├── tenant_db/       # 数据底座
│   │   │   ├── views.py    # 商品/库存/订单/客户/流向
│   │   │   └── urls.py
│   │   ├── model_gateway/   # 大模型网关
│   │   │   ├── views.py    # 模型列表/测试/部署
│   │   │   └── urls.py
│   │   ├── platform_gateway/ # 第一层预留 ( 商户数据接入 )
│   │   │   ├── views.py
│   │   │   └── urls.py
│   │   ├── tenant_ext/      # 租户扩展功能
│   │   │   ├── views.py    # 知识库/素材/任务/积分/技能/SaaS/连接器
│   │   │   └── urls.py
│   └── middleware/
│       └── tenant.py        # X-Tenant-ID 中间件
└── ai-employee-app/        # 第三层：YesGo 前端 ( Electron + React )
    └── electron/
```

```

├── main.cjs          # Electron 主进程
├── preload.cjs       # 预加载脚本
├── src/
│   ├── main.tsx      # React 入口
│   ├── App.tsx       # 根组件 (路由 + 对话派发)
│   ├── types.ts      # 全局类型定义
│   ├── index.css     # 全局样式 + CSS 变量主题
│   ├── components/   # 23 个功能组件
│   │   ├── AgentOfficeView.tsx  # AI 办公室 (核心视图)
│   │   ├── AgentConfigPanel.tsx # 智能体配置面板
│   │   ├── OfficePanel.tsx      # 办公室面板卡片
│   │   ├── RabbitOfficeScene.tsx # 白兔场景装饰
│   │   ├── RabbitHead.tsx       # 白兔头像 (统一组件)
│   │   ├── ChatView.tsx         # 对话工作台
│   │   ├── WelcomeScreen.tsx    # 欢迎页
│   │   ├── InputBar.tsx        # 输入框
│   │   ├── Sidebar.tsx         # 左侧导航栏
│   │   ├── TitleBar.tsx        # 顶栏
│   │   ├── CommandPalette.tsx  # ⌘ 命令面板
│   │   ├── DataView.tsx       # 数据看板
│   │   ├── DataBaseView.tsx    # 数据底座
│   │   ├── ClientsView.tsx     # 客户/B2B
│   │   ├── KnowledgeView.tsx   # 知识库
│   │   ├── SkillsView.tsx      # 技能市场
│   │   ├── ModelsView.tsx      # 模型网关
│   │   ├── CreditsView.tsx     # 算力积分
│   │   ├── ConfigView.tsx      # 配置中心
│   │   ├── PermissionsView.tsx # 权限管理 (RBAC)
│   │   ├── TasksView.tsx       # 任务管理
│   │   ├── MediaView.tsx       # 素材管理
│   │   └── SettingsView.tsx    # 设置
│   ├── lib/
│   │   ├── api.ts             # 统一 HTTP 客户端 (Bearer Token + X-Tenant-ID)
│   │   ├── backend.ts         # 后端集成层 (自动登录 + 同步 + CRUD helpers)
│   │   ├── dispatch.ts        # 意图识别 + 派发规则引擎
│   │   ├── dify.ts            # Dify 集成层 (5 个工作流独立 API Key)
│   │   ├── constants.ts       # AgentCode 枚举、租户套餐类型
│   │   └── theme.tsx          # 深/浅色 + 5 品牌色主题系统
│   ├── store/
│   │   └── appStore.tsx       # 全局状态中枢 (useReducer)
│   └── data/
│       ├── mockAgents.ts      # 6 智能体定义
│       └── mockSaaS.ts        # 医药 SaaS mock 数据 (向后兼容本地降级)
├── package.json
├── vite.config.ts
├── tailwind.config.js
├── tsconfig.json
└── .npmrc                  # 国内镜像 (npmmirror)

```

5. 第二层：天网大脑后端

5.1 Django App 模块一览

App	职责	API 前缀	关键端点
platform	认证 + 商户管理 + 配置	<code>`/api/v1/auth/*`</code> <code>`/api/v1/tenant`</code>	login, me, tenant info/members/role
chat	智能体对话	<code>`/api/v1/chat/*`</code>	send (消息发送+意图识别+派发+LLM)
dashboard	经营看板	<code>`/api/v1/dashboard/*`</code>	overview, kpi, alerts
tenant_db	数据底座	<code>`/api/v1/data/*`</code>	products, inventory, orders, custom
model_gateway	大模型网关	<code>`/api/v1/models/*`</code>	list, test, deploy
platform_gateway	第一层预留	<code>`/api/v1/platform/*`</code>	商品/库存/订单/客户/流向 数据同步
tenant_ext	租户扩展功能	<code>`/api/v1/*`</code>	知识库/素材/任务/积分/技能/SaaS/连接器

5.2 数据库设计（生产环境 PostgreSQL）

```
PostgreSQL
├─ public schema ( 公共数据库 )
│  └─ public_products      # 共享商品库
│  └─ public_suppliers     # 共享供应商
│  └─ public_categories    # 共享分类
├─ tenant_{credit_code}_1 ( 商户 1 )
│  └─ products             # 商户商品
│  └─ inventory            # 商户库存
│  └─ orders               # 商户订单
│  └─ customers            # 商户客户
│  └─ distribution         # 商户流向
│  └─ academic_materials   # 商户学术素材
├─ tenant_{credit_code}_2 ( 商户 2 )
│  └─ ... ( 同上结构 )
├─ platform ( 平台管理 )
│  └─ tenants              # 商户注册信息
│  └─ tenant_users         # 商户员工账号
│  └─ tenant_roles         # 角色权限
│  └─ tenant_credits       # 积分账本
│  └─ model_configs        # 大模型配置
└─ api_keys               # API 密钥
```

5.3 多租户中间件

- 所有 API 请求通过 HTTP Header ``X-Tenant-ID`` 识别商户身份 - 中间件自动提取并注入到 ``request.tenant_id`` - 数据库查询自动按租户过滤

5.4 大模型网关（预留架构）

```
大模型网关 ( Model Gateway )
├─ 商业 LLM
│  └─ OpenAI ( GPT-4o / GPT-4o-mini )
│  └─ 文心一言 ( ERNIE )
│  └─ 通义千问 ( Qwen-Max / Qwen-Plus )
│  └─ 混元 ( HunYuan-Pro )
│  └─ Claude 3.5 Sonnet
│  └─ DeepSeek ( API )
├─ 开源 LLM
│  └─ Qwen2.5-72B ( 本地部署 )
│  └─ DeepSeek-V3 ( 本地部署 )
│  └─ Llama-3.1-70B ( 本地部署 )
```

- └─ ChatGLM4-9B (本地部署)
- └─ 路由规则
 - └─ 意图识别 → 速度快、成本低的模型
 - └─ 内容生成 → 质量高、上下文大的模型
 - └─ 敏感数据 → 本地部署模型 (不出网)

6. 第三层：多商户前端

6.1 全局状态管理 (appStore)

统管以下数据领域，通过 `React Context + useReducer` 跨视图共享：

数据域	内容
agents	6 个智能体状态 (启停开关、状态、进度)
creditBalance / creditLedger	算力积分余额与消耗明细账本
models	9 个模型 (商用 5 + 开源 4) 含部署状态
knowledge	知识文档 (含绑定智能体、私有隔离标识)
media	素材资产 (图片/视频/文档)
tasks	自动任务 (定时执行配置)
saas	5 个 SaaS 连接 (双向回写开关、同步状态)
configs	每智能体双层配置 (出厂只读 + 企业自定义)
difyConfig	Dify 5 工作流独立 API Key 配置
tenant	租户信息/成员/角色/套餐
dataBaseConnectors	可链接系统 (连接器开关)
pendingTask / lastResult	跨视图任务通道 (对话→办公室→回传)
installedSkills	已安装技能列表
backendConnected	后端连接状态标志

6.2 意图识别引擎 (dispatch.ts)

双层调度架构：

```

用户输入文本
├─ 第 1 层：Dify 意图识别 → 返回 agentId + intent + confidence
│   ( 当前 Mock，预留真实 API 接缝 )
└─ 第 2 层：本地规则引擎 ( 关键词 + 实体抽取 )
    → 回退策略：强兜底
  
```

****规则引擎关键词表**：**

智能体	意图	命中关键词
purchase	采购补货	采购、补货、进货、下单、订购、买、缺货、备货
crm	客户跟进	客户、跟客、回访、药店、诊所、沟通、拜访
ops	经营分析	促销、活动、定价、比价、经营、分析、销量、报表
flow	流向监控	窜货、流向、滞销、预警、渠道、跨区域、异常
academic	学术内容	学术、合规、课件、培训、素材、患教、推广

兜底策略：关键词无命中但有商品名 → 默认派发采购智能体。

6.3 主题系统

```
ThemeProvider ( React Context )
├─ mode: 'dark' | 'light'    → body.theme-dark / body.theme-light
└─ colorTheme: 5 色         → indigo / emerald / rose / amber / cyan

CSS 变量 ( RGB 通道 )      Tailwind 颜色令牌
:root { --bg-base: 11 13 18 } → bg: { base: 'rgb(...)' }
body.theme-light { override } → 透明度修饰符正确编译
body.color-* { accent }      → 渐变 from-accent 正确编译

持久化：localStorage('yesgo-theme')，重启应用后保留。
```

7. 智能体体系

7.1 6 个智能体

AgentCode	名称	Emoji	职责
control	中控 A Agent		全局调度中枢：意图识别→派发→结果汇总
ops	运营智能体		经营分析 / 促销测算 / 比价定价 / 报表
crm	跟客智能体		客户自动跟进 / 话术生成 / 沟通台账
purchase	采购智能体		补货方案 (最快/最优/均衡) + 回写 SaaS 订单
flow	流向智能体		窜货监控 / 渠道预警 / 滞销检测
academic	学术智能体		合规学术素材生成 / 患教内容 / 分层定制

每个智能体具备： - `enabled`：启停开关 - `status`：idle / working / done - `progress`：0-100% -
`credits`：消耗积分 - `log`：最近执行摘要

7.2 对话驱动调度机制

```
用户输入指令 ( 对话视图 / 办公室输入条 )
├─ dispatch.ts 意图识别 + 智能体派发
├─ 先回确认消息 ( "好的老板，马上落实" )
├─ 写入 pendingTask 到 appStore
├─ AgentOfficeView 监听 pendingTask → runTask()
└─ → 分层日志 ( control/agent/saas/credit )
```



```

| → 进度动画 0%→100%
| → 扣算力积分 → 写入全局账本
|
├─ 执行完毕 → 回写 result 到 appStore
|
└─ App.tsx 监听 lastResult → 追加结果回报消息到对话

```

8. 视图功能清单

8.1 AI 办公室 (AgentOfficeView) ★ 核心视图

区域	功能
自然语言输入条	输入指令 → Enter 派发
中控 A 卡片	调度中状态、最近日志摘要
5 个智能体工位卡片	拟人动效、进度条、算力积分、启停开关
演示任务流按钮	一键运行补货演示
采购闭环结果卡	三套方案 (最快/最优/均衡) , 含供应商/到货天数/预估价格, 一键采纳
分层执行日志面板	四级日志 (中控/智能体/SaaS/算力) + 过滤按钮

8.2 对话工作台 (ChatView + WelcomeScreen + InputBar)

功能	说明
欢迎页	4 个快捷指令按钮 (补货/查窜货/经营分 ◆◆/客户跟进)
三种消息样式	普通消息 / 派发卡片 / 结果回报
两步回复流程	先人话确认 → 智能体派发 → 活干完自动汇报
⌘K 命令面板	全局搜索+视图导航+指令快捷输入
自适应输入框	最多 6 行自动滚动, 支持 Shift+Enter

8.3 数据看板 (DataView)

- 经营概览卡片 (营收/订单/客户/库存/智能体 5 指标) - 天网大脑实时数据面板 (后端连接时显示) - KPI 达成率 - 实时预警

8.4 数据底座 (DataBaseView)

- 可链接系统 / 已链接系统 / 未链接系统统计卡片 - 可链接系统列表 (系统名称、说明、图标、API 连接状态) - 管理员: 发布系统 / 恢复默认 / 编辑 / 删除按钮 - localStorage 持久化

8.5 客户 / B2B (ClientsView)

- 5 个 SaaS 数据源连接卡片 (状态 + 最近同步 + 双向回写) - 客户列表 (来自 CRM 数据底座) - SaaS 授权入口

8.6 知识库 (KnowledgeView)

- 多格式文档 (PDF/DOC/XLS/PPT/MD) - 搜索过滤 + 删除 - 私有隔离标签 + 绑定智能体展示 - 上传区域

8.7 技能市场 (SkillsView)

- 已安装 + 市场技能卡片网格 - 安装/卸载 toggle

8.8 模型网关 (ModelsView)

- 商用模型区 (5 个) + 开源模型区 (4 个) - 每家模型状态标签 (可用/部署中/未部署) - 连接测试 + 私有化部署按钮 - 每个智能体的模型绑定

8.9 算力积分 (CreditsView)

- 余额卡片 (含累计消耗统计) - 充值套餐 (5 档) - 按智能体消耗占比进度条 - 消耗明细账本

8.10 配置中心 (ConfigView)

- 双层配置 : 出厂默认 (只读) vs 企业自定义 (可编辑) - 每智能体 : 绑定模型 / temperature / 超时重试 / 备用模型 / 人工接管阈值 - Dify 5 工作流独立 API Key 配置 - 恢复出厂默认

8.11 权限管理 (PermissionsView · RBAC)

- 4 角色 (admin/operator/member/viewer) + 角色权限矩阵 - 6 成员 mock + CRUD - 角色-智能体绑定 + 成员个人积分账户 - 聊天时校验权限 + 积分

8.12 任务管理 (TasksView)

- 自动任务创建/开关/删除 - 执行历史

8.13 素材管理 (MediaView)

- 图片库搜索/上传/删除

8.14 设置 (SettingsView)

- 外观 (深浅色 + 5 品牌色) - 模型配置 / 安全与权限 / 关于 - 快捷导航到对应功能视图

9. 数据模型

9.1 医药 SaaS 数据底座

实体	数量	说明
商品 (Product)	15	阿莫西林、头孢克肟等常用药品
库存 (Inventory)	40	6 个仓库，含大量低于安全库存记录
供应商 (Supplier)	5	九州通 3 仓 + 华润 + 上药
采购订单 (Order)	12	含状态标签 (done/processing/pending)
流向记录 (Flow)	8	含 3 条窜货异常
客户 (Customer)	12	药店/诊所，含区域和评分

9.2 知识库文档

实体	数量	说明
知识文档 (KnowledgeDoc)	8	PDF/DOC/XLS/PPT/MD，绑定智能体，私有隔离

9.3 模型

类型	数量	说明
商用模型	5	通义千问-Max / 混元-Pro / GPT-4o / Claude
开源模型	4	Qwen2.5-72B / DeepSeek-V3 / Llama-3

9.4 SaaS 连接器

连接器	状态	说明
莱芬享 B2B	已连接	客户已有 SaaS
药企 ERP	已连接	用友 U8
天猫旗舰店	待授权	B2C 平台
京东大药房	待授权	B2C 平台
拼多多医药	待授权	三方平台

9.5 可链接系统 (DataBaseConnectors)

序号	系统名称	类型
1	莱芬享 B2B 供应链	B2B
2	用友 U8 ERP	ERP
3	金蝶云星空	ERP
4	SAP Business One	ERP
5	天猫医药馆	B2C
6	京东大药房	B2C
7	药智网数据平台	三方
8	米内网数据	三方
9	九州通物流 WMS	三方

10. API 接口设计

10.1 统一规范

- 基础 URL** : `http://localhost:8000/api/v1` - **认证方式** : `Authorization: Bearer {JWT-Token}` - **多租户** : `X-Tenant-ID: {tenant\_id}` - **统一响应格式** :

```
{ "code": 0, "msg": "ok", "data": { ... } }
```

10.2 认证模块 (`/api/v1/auth/\*`)

方法	路径	说明
POST	/auth/login	用户登录，返回 JWT Token
GET	/auth/me	获取当前用户信息

10.3 租户管理 (`/api/v1/tenant/\*`)

方法	路径	说明
GET	/tenant/info	商户信息
GET	/tenant/members	成员列表
POST	/tenant/members/create	创建成员
PUT	/tenant/members/{id}	更新成员
DELETE	/tenant/members/{id}/delete	删除成员
GET	/tenant/roles	角色列表
POST	/tenant/roles/create	创建角色
PUT	/tenant/roles/{id}	更新角色
DELETE	/tenant/roles/{id}/delete	删除角色
GET	/tenant/package	套餐信息
GET	/tenant/agents	智能体列表

10.4 智能体对话 (`/api/v1/chat/\*`)

方法	路径	说明
POST	/chat/send	发送消息 (意图识别+派发+LLM)
GET	/chat/history	对话历史
GET	/chat/conversations	会话列表

10.5 数据底座 (`/api/v1/data/\*`)

方法	路径	说明
GET	/data/products	商品列表
GET	/data/inventory	库存数据
GET	/data/orders	订单数据
GET	/data/customers	客户数据
GET	/data/distribution	流向数据

10.6 经营看板 (`/api/v1/dashboard/\*`)

方法	路径	说明
GET	/dashboard/overview	总览数据 (营收/订单/客户/库存/智能体)
GET	/dashboard/kpi	KPI 达成率
GET	/dashboard/alerts	实时预警

10.7 模型网关 (`/api/v1/models/\*`)

方法	路径	说明
GET	/models/list	可用模型列表
POST	/models/test	模型连接测试
POST	/models/deploy	部署模型

10.8 系统配置 (`/api/v1/config/\*`)

方法	路径	说明
GET	/config/	获取系统配置
PUT	/config/	更新系统配置
GET	/config/dify	获取 Dify 工作流配置
PUT	/config/dify	更新 Dify 工作流配置

10.9 租户扩展 (`/api/v1/\*`)

方法	路径	说明
知识库		
GET	/docs	知识库文档列表
POST	/docs	创建文档
DELETE	/docs/{doc_id}	删除文档
素材		
GET	/assets	素材列表
POST	/assets	创建素材
DELETE	/assets/{asset_id}	删除素材
任务		
GET	/tasks	任务列表
POST	/tasks	创建任务
PUT	/tasks/{task_id}	更新任务
DELETE	/tasks/{task_id}/delete	删除任务
积分		
GET	/credits/balance	积分余额
GET	/credits/ledger	积分流水
POST	/credits/recharge	积分充值
技能		
GET	/skills/list	技能列表
POST	/skills/toggle	切换技能状态
SaaS		
GET	/saas/connections	SaaS 连接列表
PUT	/saas/connections/{conn_id}	更新连接状态
连接器		
GET	/connectors	连接器列表
POST	/connectors	创建连接器
PUT	/connectors/{conn_id}	更新连接器
DELETE	/connectors/{conn_id}/delete	删除连接器

10.10 第一层预留 (`/api/v1/platform/\*`)

方法	路径	说明
POST	/platform/products/sync	商品同步
POST	/platform/inventory/sync	库存同步
POST	/platform/orders/sync	订单同步
POST	/platform/customers/sync	客户同步
POST	/platform/distribution/sync	流向同步

11. 数据通讯闭环

11.1 启动流程

```

前端启动
├── StoreProvider mount → useEffect
├── autoLogin() → 获取 JWT Token
├── syncAllFromBackend() → 并行拉取 7 个核心端点
│   ├── tenant info
│   ├── members
│   ├── roles
│   ├── package
│   ├── agents
│   ├── models
│   └── config / dify
├── syncExtendedFromBackend() → 并行拉取 8 个扩展端点
│   ├── knowledge docs
│   ├── media assets
│   ├── tasks
│   ├── credits balance/ledger
│   ├── skills
│   ├── SaaS connections
│   └── connectors
└── dispatch SYNC_FROM_BACKEND → 更新全局 store
  
```

11.2 CRUD 操作模式

```

用户操作 ( CRUD )
├── 视图组件检查 backendConnected
├── 后端可用 → 调用 backend.ts CRUD helper → POST/PUT/DELETE 后端 API
│   ├── 成功 → 更新本地 store
│   └── 失败 → 保持本地不变 + 显示错误
└── 后端不可用 → 降级到本地 store 操作
  
```

11.3 验证状态 (2026-07-24)

- ****27 个 API 端点全部通过** (HTTP 200) - ****11 项 CRUD 操作全部验证通过**** -**
- 知识库、素材、任务、积分、技能、连接器、成员、角色、模型、配置、SaaS 均有完整的读写闭环 -
- TypeScript 编译 0 错误，生产构建成功

12. 构建与部署

12.1 后端启动

```
cd yesgo-backend
/Users/chenshenghe/.workbuddy/binaries/python/envs/yesgo-backend/bin/python3 manage.py runserver 0.0.0.0:8000
```

12.2 前端开发

```
cd ai-employee-app
npm run dev # Vite dev server (port 5180)
```

12.3 前端构建

```
npm run build:vite # tsc + vite build → dist/
```

12.4 打包

```
# macOS DMG
ELECTRON_RUN_AS_NODE= CSC_IDENTITY_AUTO_DISCOVERY=false \
ELECTRON_MIRROR=https://npmmirror.com/mirrors/electron/ \
npm run build:mac

# Windows NSIS
npm run build:win
```

****关键环境变量****：

- 必须清空 ``ELECTRON_RUN_AS_NODE`` (WorkBuddy 沙箱默认设为 `1`)
- ``CSC_IDENTITY_AUTO_DISCOVERY=false`` (跳过代码签名)
- ``ELECTRON_MIRROR`` 指向国内镜像 (npmmirror)

12.5 npm 镜像配置 (.npmrc)

```
registry=https://registry.npmmirror.com
electron_mirror=https://npmmirror.com/mirrors/electron/
electron_builder_binaries_mirror=https://npmmirror.com/mirrors/electron-builder-binaries/
```

13. 当前状态与路线图

13.1 已完成

阶段	内容	状态	版本
P1	虚拟办公室 (工位可视化 + 中空调		v0.1.0
P2	业务闭环 (对话派发 + 采购闭环 +		v0.2.0
P3	平台能力 (模型网关/算力积分/配		v0.2.0
P4	对话体验升级 (自然语言应答 + 派		v0.2.0
P5	端到端逻辑跑通 + Bug 修复 + 跨视		v0.2.1
P6	天网大脑 Django 后端搭建 (8 模		v0.3.0
P7	前后端数据通讯闭环 (所有视图接		v0.3.0
P8	RBAC 权限管理 + AI 办公室配置中		v0.3.0
P9	数据底座持久化 + 后台发布 CRUI		v0.3.0

13.2 待用户校验

模块	说明
对话	消息发送-意图识别-智能体派发-L回复 全链路
数据看板	商品/库存/订单/客户/流向数据展示
知识库	文档上传/删除/列表
技能	技能列表/安装切换
模型	模型列表/部署/连接测试
积分	余额/流水/充值
权限	角色/成员 CRUD
任务	自动任务创建/开关/删除
素材	图片上传/删除/搜索
客户	客户列表/SaaS 授权/双向打通
数据底座	连接器 CRUD
配置	系统配置/Dify 工作流配置

13.3 待接真

位置	说明
大模型 API	替换 dispatch 规则引擎为真实 LLM function-ca
Dify 工作流	5 个智能体工作流真实 API 对接
SaaS 数据	替换 mockSaaS 为真实 SaaS OpenAPI 调用
算力计费	接入真实算力计费网关
对象存储	接入阿里云 OSS (素材上传)
消息队列	接入 Kafka (异步任务)
数据库	SQLite → PostgreSQL 迁移
代码签名	macOS / Windows 代码签名

附录 A：关键术语

术语	含义
天网大脑	第二层中台总控 (Django 后端)
数据底座	商户业务数据 (商品/库存/订单/客户/流向)
中控 A	统筹调度中枢智能体
双层调度	Dify (线上) + 本地规则引擎 (回退)
AgentCode	智能体编码标识 (procurement/operations/crm/
X-Tenant-ID	多租户隔离请求头

附录 B：已知限制

- ****沙箱环境无法启动 GUI****：只能打包成 dmg 让用户本地安装 2. ****未代码签名****：dmg 首次打开需系统设置允许 3. ****开发环境 SQLite****：生产需切 PostgreSQL 4. ****数据全 Mock****：后端返回的是结构化 mock 数据，接真实 LLM/SaaS 可立即跑真实业务

文档维护：本文档随平台开发持续更新。最新版本请查看版本号标注。